

Spatial Analysis in R for Forest Carbon Verifiers

A Practical GIS Training for Forest Carbon Audits & and Remote Sensing

Clone the full repo and ebook here <https://github.com/seamusrobertmurphy/training-rspatial>

Murphy, S; 2026-07-17

Table of contents

Preface	3
1. Getting Started	4
1.1 Reproducibility	4
1.2 The stack.....	4
1.3 Projects	5
1.4 The running dataset.....	5
1.5 Vector and raster	7
1.6 A first map.....	7
1.7 First checks.....	8
1.8 Exercises	9
2. Vector Data.....	9
2.1 Geometry types	9
2.2 sf anatomy	9
2.3 Building polygons.....	10
2.4 Spatial joins.....	10
2.5 Support.....	11
2.6 Keystone: area checks.....	12
2.7 Exercises	13
3. Raster Data	13
3.1 The raster model.....	13
3.2 Inspecting rasters	14
3.3 Raster algebra	14
3.4 Reclassification	15
3.5 Extraction.....	15
3.6 Keystone: land-cover reclassification	16
3.7 Exercises	17
4. Terrain and Hydrology	17
4.1 Elevation	17
4.2 Terrain derivatives.....	18
4.3 Flow direction.....	19
4.4 Keystone: hydrological analysis.....	19
4.5 Exercises	20
5. LiDAR and Point Clouds	20
5.1 Point clouds	20

5.2 Ground and canopy.....	21
5.3 Building a CHM	22
5.4 Tree detection	22
5.5 Keystone: lidar-forestry	23
5.6 Exercises	23
6. Disturbance and Risk.....	23
6.1 Change detection	23
6.2 Accuracy assessment	25
6.3 Spatial risk	25
6.4 Keystone: disturbance checks and VM0048 risk.....	26
6.5 Exercises	26
7. Uncertainty.....	27
7.1 Why uncertainty.....	27
7.2 A carbon estimate.....	27
7.3 Sampling error.....	27
7.4 Monte Carlo propagation	28
7.5 Conservative estimates	29
7.6 Keystone: area-adjusted accuracy.....	30
7.7 Exercises	30
8. Time Series	30
8.1 Data cubes.....	30
8.2 Temporal reduction.....	31
8.3 Trends	32
8.4 Detecting the event	32
8.5 Keystone: IPCC Tier 1 fire emissions	32
8.6 Exercises	33
9. Spatial Patterns.....	34
9.1 Point patterns.....	34
9.2 Spatial randomness	34
9.3 Intensity	35
9.4 Nearest neighbour	35
9.5 Ripley's K	35
9.6 Keystone: seedlings	36
9.7 Exercises	36
Appendix A — Automation, Protocols and Registries	37
A.1 Beyond the analysis	37
A.2 Reproducible runtimes.....	37
A.3 Data protocols and QA/QC	37
Appendix B — References.....	38

Preface

This is a practical, comprehensive introduction to geospatial data science with R, built for scientists, analysts, and auditors who need to move confidently from a stack of shapefiles and satellite imagery to a defensible result. It teaches the foundations every analyst of spatial data must understand, how spatial data are represented, referenced, measured, combined, and monitored through time, using free and open tools, and it assumes some statistics but no prior GIS.

The course is organised as nine short chapters that run from simple to complex. Each teaches a foundational concept on one running dataset, then closes with a worked example drawn from a real forest-carbon verification project, so the concept earns its example and the example proves the concept. By the end you will have carried a single forest plot from raw coordinates through vector and raster analysis, terrain, LiDAR, disturbance detection, and uncertainty, and worked two capstones: a time-series analysis that ends in a quantified emission, and a spatial-pattern analysis of forest regeneration.

The chapters

1. Getting Started — reproducible R projects, the spatial stack, vector versus raster, a first map and coordinate check.
2. Vector Data — geometries, attributes and support, spatial joins; closing on an area check.
3. Raster Data — grids, map algebra, reclassification, extraction; closing on land-cover reclassification.
4. Terrain and Hydrology — elevation, slope and aspect, flow; closing on inundation mapping.
5. LiDAR and Point Clouds — returns, ground and canopy, tree detection; closing on a LiDAR forest inventory.
6. Disturbance and Risk — change detection, accuracy assessment, risk modelling; closing on VM0048 risk and ACR accuracy.
7. Uncertainty — sampling error, Monte Carlo propagation, conservative estimates; closing on area-adjusted accuracy.
8. Time Series (capstone I) — space-time cubes, trends, change to emissions; closing on IPCC Tier 1 fire emissions.
9. Spatial Patterns (capstone II) — point patterns, intensity, nearest-neighbour, Ripley's K; closing on post-fire conifer regeneration.

An appendix covers the surrounding discipline: automation, spatial-data protocols, and registry practice.

The running dataset

Every chapter draws on one dataset: a mapped forest-dynamics plot modelled on the Smithsonian ForestGEO census at the Conservation Biology Institute in Front Royal, Virginia. It records 2,287 stems in a 400 by 640 metre plot, each identified to species and measured for diameter. A forestry dataset is incidental to the methods, which are general to all spatial data, but it gives the examples a consistent grounding and lets the applied exercises connect to real problems in inventory, carbon, and monitoring.

 About the data

The stem list (species and diameter) follows the published ForestGEO census used in the companion uncertainty book. The within-plot coordinates were generated across the real plot footprint because the source census table omitted them, so the maps are representative rather than a survey of record. Terrain, imagery, LiDAR, and time-series layers are simulated over the same footprint so the book renders offline and reproducibly, and every synthetic layer says so where it is built. The keystone closings describe workflows from real verification projects and reproduce their analytical core on this shared data. The one exception is the second capstone (Chapter 9), which runs on the real, published coordinates of a post-fire conifer regeneration survey in the Selkirk Mountains of British Columbia (Murphy et al., 2026).

Using this book

Each chapter is a self-contained lesson that builds on the last. Code is shown in full and is meant to be run; the fastest way to learn is to work through it line by line and then attempt the exercises. Readers who want a deeper or more general treatment are well served by the canonical texts in the field (Bivand et al., 2013; Lovelace et al., 2019; Pebesma & Bivand, 2023), and the `sf` and `terra` packages that underpin the modern R-spatial stack are described by Pebesma (2018) and Hijmans (2024). Where the applied material touches forest-carbon conventions it follows IPCC (2006).

1. Getting Started

1.1 Reproducibility

An audit stands or falls on whether someone else can reproduce the number. A spreadsheet built with forty manual clicks cannot be re-run, cannot be reviewed line by line, and cannot be trusted a year later when the inputs have changed. A script can. This is the reason to do spatial work in R: every decision, from the projection you chose to the buffer distance you applied, is written down and repeatable.

R is a free, open-source language with a mature ecosystem for geographic data. In one session you can read a shapefile a surveyor sent you, pull an elevation model from an open server, overlay the two, extract terrain at your field plots, fit a model, and export a map, without leaving the environment or paying for a licence. For a forest-carbon or agriculture auditor that means the whole chain from raw data to reported estimate lives in one reviewable document. This course uses the modern R-spatial stack, built on two packages that have become the standard tools of the field (Hijmans, 2024; Pebesma, 2018).

1.2 The stack

You will meet these packages throughout the book. Install them once before you begin.

Package	Role
<code>sf</code>	Vector data: points, lines, polygons
<code>terra</code>	Raster data: imagery, elevation, gridded surfaces
<code>dplyr</code> / <code>tidyverse</code>	Data wrangling and pipes
<code>tmap</code>	Thematic maps, static and interactive
<code>ggplot2</code>	Statistical graphics
<code>gstat</code>	Geostatistics and interpolation

Package	Role
spdep	Spatial autocorrelation and neighbours

To install everything the course needs, run this once in the console. It is set not to execute when the book renders.

```
install.packages(c(
  "sf", "terra", "dplyr", "tidyverse", "tmap",
  "ggplot2", "gstat", "spdep", "elevatr", "rnatualearth"
))
```

Load the core three at the top of any spatial script:

```
library(sf)
library(terra)
library(dplyr)
```

The messages `sf` and `terra` print on loading name the versions of GDAL, GEOS and PROJ you are linked against. Note them: a projection or geometry result can occasionally differ between library versions.

1.3 Projects

Before loading data, put your work in an RStudio Project, a folder with an `.Rproj` marker. It fixes two things that otherwise cause endless confusion: it sets the working directory to the project root, and it lets you refer to data with short relative paths like `data/scbi_stems.csv` instead of a full path tied to your machine. A tidy layout looks like:

```
training-rspatial/
├── data/                # input datasets
├── 01-getting-started.qmd
├── 02-vector-data.qmd
└── training-rspatial.Rproj
```

With a project open, every path in this book that begins with `data/` resolves correctly.

1.4 The running dataset

One dataset runs through the whole course: a mapped forest-dynamics plot, modelled on the Smithsonian ForestGEO plot at the Conservation Biology Institute in Front Royal, Virginia. Every stem in a 400 by 640 metre plot is stem-mapped, identified to species, and measured for diameter. It is an ideal teaching set because it is large, real in structure, and every tree carries a location.

 About the data

The stem list (species, diameter) follows the published ForestGEO census used in the companion uncertainty book. The within-plot coordinates were generated across the real plot footprint because the source census table omitted them, so the map is representative rather than a survey of record. Terrain, imagery and LiDAR layers in later chapters are simulated over the same footprint for a clean, always-runnable progression, and every synthetic layer says so where it is built.

The stem table is an ordinary CSV, so base R reads it without any spatial machinery:

```
stems <- read.csv("data/scbi_stems.csv")
str(stems)
#> 'data.frame': 2287 obs. of 10 variables:
#> $ treeID : int 2695 1229 1230 1295 1229 66 2600 4936 1229 1005 ...
#> $ stemID : int 2695 38557 1230 32303 32273 31258 2600 4936 36996 1005 ...
#> $ dbh : num 1.4 1.7 1.4 1 2.5 2.2 1.8 1.4 2 1.4 ...
#> $ genus : chr "Acer" "Acer" "Acer" "Acer" ...
#> $ species: chr "negundo" "negundo" "negundo" "negundo" ...
#> $ family : chr "Sapindaceae" "Sapindaceae" "Sapindaceae" "Sapindaceae" ...
#> $ gx : num 188 122 164 106 325 ...
#> $ gy : num 466 568 459 157 319 ...
#> $ lon : num -78.1 -78.1 -78.1 -78.1 -78.1 ...
#> $ lat : num 38.9 38.9 38.9 38.9 38.9 ...
```

Two functions tell you almost everything about a new dataset. `summary()` shows the distribution of each column; a grouped `dplyr` summary reads the forest:

```
summary(stems[, c("dbh", "gx", "gy")])
#>      dbh      gx      gy
#> Min.   : 1.00   Min.   : 0.2   Min.   : 0.0
#> 1st Qu.: 1.20   1st Qu.: 99.1   1st Qu.:160.9
#> Median : 1.70   Median :202.4   Median :321.7
#> Mean   : 5.52   Mean   :201.5   Mean   :322.4
#> 3rd Qu.: 3.40   3rd Qu.:302.0   3rd Qu.:481.4
#> Max.   :92.00   Max.   :400.0   Max.   :639.9

stems |>
  group_by(genus) |>
  summarise(n = n(), mean_dbh = round(mean(dbh), 1)) |>
  arrange(desc(n)) |>
  head(8)
#> # A tibble: 8 × 3
#>   genus      n mean_dbh
#>   <chr>   <int>   <dbl>
#> 1 Linder  1201     1.6
#> 2 Rosa    151     1.5
#> 3 Carya   137    12.7
#> 4 Amelanchier 123     3.1
#> 5 Hamamelis 107     2.6
#> 6 Quercus   84    27.8
#> 7 Fraxinus  76    23.6
#> 8 Carpinus  57     5.4
```

Diameters run from saplings at 1 cm to canopy trees near a metre, and a handful of genera dominate the stem count. That is the size and composition structure every biomass and carbon calculation later in the book will rest on.

1.5 Vector and raster

Spatial data come in two great families, and knowing which you hold is the first decision in any analysis. Vector data represent discrete objects by their geometry: a tree is a point, a road a line, a stand boundary a polygon, each carrying attributes in a table. Raster data represent a continuous surface as a grid of equal cells, each holding one value: elevation, reflectance, canopy height. Our stems are vector; the elevation and imagery we bring in later are raster. Most real analysis moves between the two, and the middle chapters of this book are about doing that cleanly.

1.6 A first map

The CSV knows nothing about geography yet: `lon` and `lat` are just numbers. `sf` turns them into geometry by naming the coordinate columns and the coordinate reference system, here WGS 84 longitude/latitude, whose EPSG code is 4326:

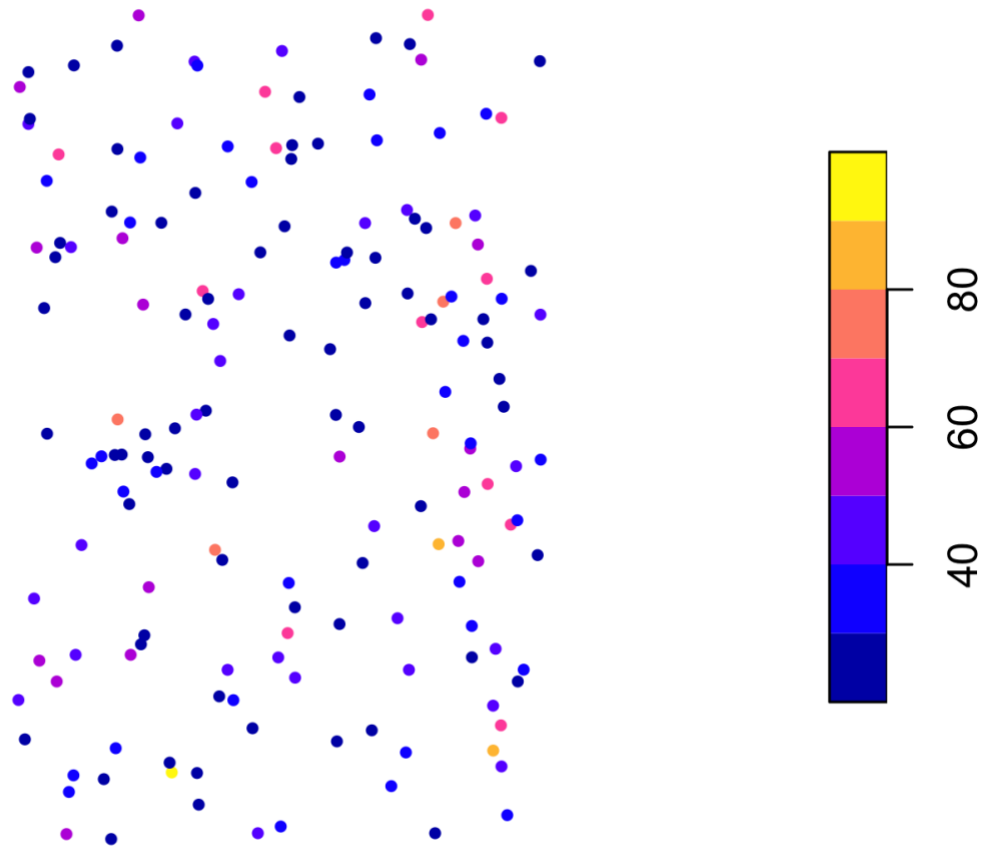
```
stems_sf <- st_as_sf(stems, coords = c("lon", "lat"), crs = 4326)
stems_sf
#> Simple feature collection with 2287 features and 8 fields
#> Geometry type: POINT
#> Dimension: XY
#> Bounding box: xmin: -78.1454 ymin: 38.8935 xmax: -78.14078 ymax: 38.89925
#> Geodetic CRS: WGS 84
#> First 10 features:
#>   treeID stemID dbh genus species family gx gy
#> 1 2695 2695 1.4 Acer negundo Sapindaceae 188.0 466.1
#> 2 1229 38557 1.7 Acer negundo Sapindaceae 121.5 567.9
#> 3 1230 1230 1.4 Acer negundo Sapindaceae 164.0 458.6
#> 4 1295 32303 1.0 Acer negundo Sapindaceae 106.1 156.9
#> 5 1229 32273 2.5 Acer negundo Sapindaceae 325.0 318.9
#> 6 66 31258 2.2 Acer negundo Sapindaceae 166.3 465.8
#> 7 2600 2600 1.8 Acer negundo Sapindaceae 385.3 198.1
#> 8 4936 4936 1.4 Acer negundo Sapindaceae 281.6 332.4
#> 9 1229 36996 2.0 Acer negundo Sapindaceae 292.5 639.8
#> 10 1005 1005 1.4 Acer negundo Sapindaceae 82.6 481.7
#>   geometry
#> 1 POINT (-78.14323 38.89769)
#> 2 POINT (-78.144 38.8986)
#> 3 POINT (-78.14351 38.89762)
#> 4 POINT (-78.14418 38.89491)
#> 5 POINT (-78.14165 38.89637)
#> 6 POINT (-78.14348 38.89768)
#> 7 POINT (-78.14095 38.89528)
#> 8 POINT (-78.14215 38.89649)
#> 9 POINT (-78.14202 38.89925)
#> 10 POINT (-78.14445 38.89783)
```

Notice what `sf` prints: the geometry type (`POINT`), the bounding box, the CRS (Geodetic CRS: WGS 84), and the attribute table with a `geometry` column on the end. An `sf` object is a data frame that also knows where each row sits on Earth. The plot layer also ships as a shapefile, read the same way with `st_read("data/scbi_stems.shp")`.

Plotting one attribute draws a map coloured by that value. We map the larger stems to keep the picture legible:

```
canopy <- stems_sf[stems_sf$dbh >= 20, ]
plot(canopy["dbh"], pch = 19, cex = 0.5, main = "Stems ≥ 20 cm DBH")
```

Stems ≥ 20 cm DBH



There is the plot: canopy trees scattered across the footprint, shaded by diameter. From here the course is about doing this properly.

1.7 First checks

The first thing to do on any real project is not analysis but a coordinate sanity check. A layer whose CRS is wrong, or missing, will silently misplace every downstream result, and the error is invisible on a plot that looks fine on its own axes. Two questions answer it: does the layer declare a CRS, and does it land where it should on Earth?

```
st_crs(stems_sf)$epsg           # 4326: the layer knows its CRS
#> [1] 4326
st_bbox(stems_sf)               # does the extent sit where you expect?
#>      xmin      ymin      xmax      ymax
#> -78.14540  38.89350 -78.14078  38.89925
```

The bounding box should fall over Front Royal, Virginia (near 78.1° W, 38.9° N). If it landed in the Gulf of Guinea at (0, 0), the coordinates were never georeferenced. This is exactly the opening move of any mapping template: read the layer, confirm its CRS, confirm its extent, and only then

begin. Every keystone project in this book starts here, because a defensible result begins with knowing your data are where they claim to be.

1.8 Exercises

1. Read `scbi_stems.csv` and report the number of stems, the number of distinct genera, and the mean and maximum diameter. Which genus has the most stems?
2. Convert the table to an `sf` object with `st_as_sf()`, then confirm its CRS with `st_crs()` and its extent with `st_bbox()`. In one sentence, why is checking the CRS the first thing you should do?
3. Map the stems coloured by genus for the three most common genera only. Do they appear mixed or spatially segregated?
4. Add a `basal_area` column (the cross-sectional area of each stem, $\pi * (dbh/2)^2$ in cm^2) and report total basal area by genus. Which genus dominates the stand by basal area rather than by stem count?

2. Vector Data

2.1 Geometry types

Vector data represent the world as discrete shapes. The simple-features standard, which `sf` implements, defines a small family of geometry types built from coordinates: a point is a single location, a linestring an ordered path, a polygon an enclosed area, and each has a *multi* form for features made of several parts. Almost everything an auditor handles is one of these: stems and plot centres are points, streams and roads are lines, stands and ownership parcels are polygons.

```
library(sf)
library(dplyr)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
                  coords = c("lon", "lat"), crs = 4326)
st_geometry_type(stems)[1]
#> [1] POINT
#> 18 Levels: GEOMETRY POINT LINESTRING POLYGON MULTIPOINT ... TRIANGLE
```

Every metric operation, area, distance, overlap, is meaningless on longitude and latitude, because a degree is not a fixed length. So the first move in vector analysis is to project to a coordinate system whose units are metres. For this plot in Virginia that is UTM zone 17 North, EPSG 32617:

```
stems <- st_transform(stems, 32617)
st_bbox(stems) # now in metres
#>      xmin      ymin      xmax      ymax
#> 747559.3 4308835.3 747976.6 4309480.1
```

2.2 sf anatomy

An `sf` object is a table whose rows are features and whose last column, of class `sfc`, holds the geometry. Because it is a data frame first and spatial second, the whole `dplyr` grammar applies and the geometry is carried along automatically:

```
stems |>
  filter(dbh >= 30) |>
  mutate(basal_area_cm2 = pi * (dbh / 2)^2) |>
  select(treeID, genus, dbh, basal_area_cm2) |>
```

```

head()
#> Simple feature collection with 6 features and 4 fields
#> Geometry type: POINT
#> Dimension: XY
#> Bounding box: xmin: 747567.3 ymin: 4308977 xmax: 747908.4 ymax: 4309385
#> Projected CRS: WGS 84 / UTM zone 17N
#>   treeID genus dbh basal_area_cm2 geometry
#> 1 1139 Acer 44.0 1520.5308 POINT (747859.4 4309329)
#> 2 2436 Acer 37.4 1098.5835 POINT (747648.9 4309107)
#> 3 1134 Acer 60.0 2827.4334 POINT (747588.6 4308977)
#> 4 1031 Acer 42.5 1418.6254 POINT (747567.3 4309385)
#> 5 3800 Acer 33.2 865.6973 POINT (747908.4 4309047)
#> 6 29 Acer 32.6 834.6898 POINT (747905.6 4309230)

```

`st_drop_geometry()` turns the layer back into an ordinary table when you want a plain summary, and `st_geometry()` extracts just the shapes when you want to plot outlines with no attribute shading.

2.3 Building polygons

Analysis usually needs areas, not just points. We divide the plot into 100-metre quadrats with `st_make_grid()`, the standard way to impose a regular sampling frame, and keep only the cells that cover the stems:

```

grid <- st_make_grid(stems, cellsize = 100) |> st_as_sf()
grid$quadrat <- seq_len(nrow(grid))
grid <- grid[lengths(st_intersects(grid, stems)) > 0, ]
nrow(grid)
#> [1] 34

```

2.4 Spatial joins

A spatial join attaches attributes by location rather than by a shared key. `st_join()` labels each stem with the quadrat that contains it, after which the counting is ordinary `dplyr`:

```

stems_q <- st_join(stems, grid, join = st_within)

per_quadrat <- stems_q |>
  st_drop_geometry() |>
  group_by(quadrat) |>
  summarise(n_stems = n(),
            mean_dbh = round(mean(dbh), 1))
head(per_quadrat)
#> # A tibble: 6 × 3
#>   quadrat n_stems mean_dbh
#>   <int>   <int>   <dbl>
#> 1     1      80     5.8
#> 2     2     100     4.8
#> 3     3     98      4
#> 4     4     76     6.6
#> 5     5      9      2
#> 6     6     62     7.4

```

This point-in-polygon join is the workhorse of inventory: it is how you assign field stems to strata, plots to ownership parcels, or detections to management units.

2.5 Support

An attribute value stands in a particular relationship to the geometry it is attached to, and getting that relationship wrong is one of the most common and least visible errors in spatial analysis. The support of an attribute is the area over which it was measured or to which it applies. A stem count has each quadrat as its support; a per-hectare density refers to a hectare. Two numbers can look identical yet mean different things because their support differs.

Attributes divide into two kinds by how they behave when a geometry changes size. An extensive attribute is a total that scales with area, a count of stems or a tonnage of carbon; split a polygon and the value divides between the pieces. An intensive attribute is a rate or density that does not scale, stems per hectare or mean diameter; split the polygon and each piece keeps the value. The distinction dictates how you aggregate: sum the extensive total, average the intensive rate.

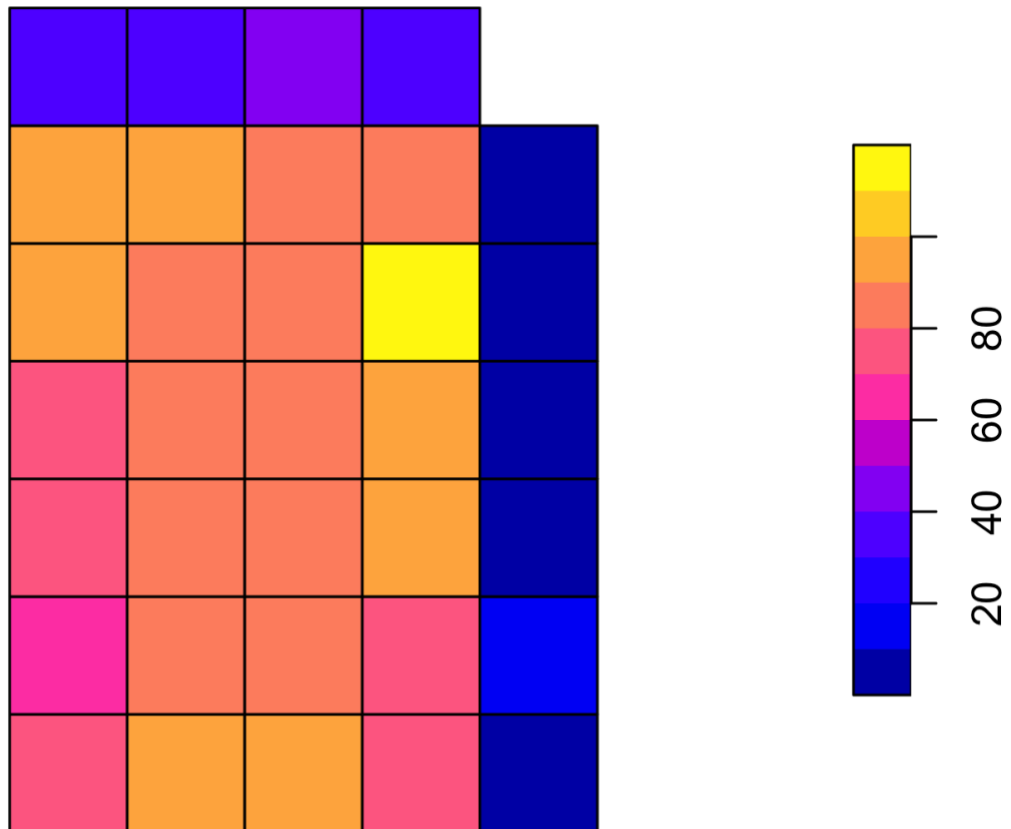
```
grid_join <- left_join(grid, per_quadrat, by = "quadrat")
grid_join$area_ha <- as.numeric(st_area(grid_join)) / 1e4      # support, in ha
grid_join$stems_ha <- grid_join$n_stems / grid_join$area_ha    # intensive

# extensive -> sum; intensive -> area-weighted mean
c(total_stems = sum(grid_join$n_stems),
  mean_density = round(weighted.mean(grid_join$stems_ha, grid_join$area_ha), 0))
#> total_stems mean_density
#>      2285          67
```

Summing a density, or averaging a total, is simply wrong, and it is exactly how reported area totals silently inflate. Always be able to state the support of a number, and never combine values of different support without an explicit rule.

```
plot(grid_join["stems_ha"], main = "Stem density (stems/ha)")
```

Stem density (stems/ha)



2.6 Keystone: area checks

The applied heart of vector GIS in verification work is the area check: confirming that the area a project *claims* matches the area its geometry actually encloses, and that its parcels do not overlap or leave gaps. Overstated area is overstated crediting, so this is often the first quantitative finding in an audit. The tools are exactly those above, `st_area`, `st_is_valid`, `st_intersection`, applied to the project's own polygons.

We stand in for the project boundary with the plot's nominal extent and check it against the mapped stems. First, geometry validity, because area and overlap operations on an invalid polygon return nonsense:

```
boundary <- st_as_sf(st_bbox(stems)) |> st_sf(id = "claimed_boundary")
all(st_is_valid(boundary))
#> [1] TRUE
```

Now the area reconciliation. A claimed area is compared with the geometry's measured area; the auditor reports the difference:

```

claimed_ha <- 25.6 # nominal 400 x 640 m plot
measured_ha <- as.numeric(st_area(boundary)) / 1e4

data.frame(
  claimed_ha = claimed_ha,
  measured_ha = round(measured_ha, 2),
  difference_ha = round(measured_ha - claimed_ha, 2),
  pct_error = round(100 * (measured_ha - claimed_ha) / claimed_ha, 1)
)
#>   claimed_ha measured_ha difference_ha pct_error
#> 1      25.6      26.9          1.3         5.1

```

Finally, overlap detection between parcels. Where two subareas are submitted, their intersection must be empty; any positive intersection area is double-counted land:

```

a <- st_as_sfc(st_bbox(c(xmin = 747000, xmax = 747250,
                        ymin = 4306000, ymax = 4306400), crs = 32617))
b <- st_as_sfc(st_bbox(c(xmin = 747200, xmax = 747450,
                        ymin = 4306000, ymax = 4306400), crs = 32617))
overlap <- st_intersection(a, b)
as.numeric(st_area(overlap)) / 1e4 # hectares of double-counted area
#> [1] 2

```

A non-zero result is a finding: those hectares are claimed twice and must be reconciled before any area total is accepted. The full project workflow layers on shapefile ingest, coordinate-system checks, and a written report, but the logic is this: validate the geometry, measure the area, compare to the claim, and test every parcel pair for overlap.

2.7 Exercises

1. Project the stems to UTM 17N and build a 50-metre quadrat grid. How many quadrats cover the plot, and how does the mean stem count per quadrat change from the 100-metre version? Relate this to support.
2. Classify `n_stems` and `stems_ha` as extensive or intensive, and state which should be summed and which averaged when quadrats are merged into blocks.
3. Take the `claimed_ha` of 25.6 and imagine the true mapped boundary were 5% smaller. Compute the over-crediting in hectares and explain why an auditor treats positive area error differently from negative.
4. (*Applied.*) Two submitted parcels overlap by 3 ha out of a claimed 40 ha total. Write the `st_intersection` and `st_area` calls that quantify the overlap, and state the corrected non-overlapping total.

3. Raster Data

3.1 The raster model

A raster represents a surface as a grid of equal cells, each holding one value. Where vector data draw discrete objects, rasters describe things that vary continuously across space, elevation, reflectance, temperature, canopy height, and the things we derive from imagery: land cover, disturbance, biomass. The data model is nothing more than a matrix plus the georeferencing that ties cell (1,1) to a place and fixes the cell size. `terra` is the modern package for it (Hijmans, 2024).

We build a raster over the study plot by summarising the stems onto a grid. First project the stems and give each a basal area, then lay down a 10-metre grid and total the basal area falling in each cell:

```
library(terra)
library(sf)

stems_sf <- st_as_sf(read.csv("data/scbi_stems.csv"),
                      coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
stems_sf$ba <- pi * (stems_sf$dbh / 2)^2 / 1e4 # basal area, m² per stem

stems <- vect(stems_sf)
r <- rast(ext(stems), resolution = 10, crs = "EPSG:32617")
#> Error:
#> ! [rast] empty srs
ba <- rasterize(stems, r, field = "ba", fun = sum, background = 0)
#> Error in `h()`:
#> ! error in evaluating the argument 'y' in selecting a method for function 'rasterize': object 'r' not found
names(ba) <- "basal_area"
#> Error:
#> ! object 'ba' not found
ba
#> Error:
#> ! object 'ba' not found
```

The printout is the raster data model made concrete: rows and columns, cell size, extent, CRS, and the value range. Each 10-metre cell now holds the total stem basal area within it, a crude but genuine biomass proxy.

3.2 Inspecting rasters

The same handful of accessors describe any raster:

```
res(ba) # cell size, metres
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'res': object 'ba' not found
ext(ba) # bounding box
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'ext': object 'ba' not found
global(ba, c("min", "max", "mean")) # value range
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'ba' not found
plot(ba, main = "Basal area per 10 m cell (m²)")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'ba' not found
```

3.3 Raster algebra

Because a raster is a grid of numbers, arithmetic on rasters is arithmetic on cells. This is map algebra, and it is why continuous surfaces are so easy to work with. Converting basal area to an aboveground-carbon proxy is one expression applied to every cell at once:

```
carbon <- ba * 12 # illustrative MgC per cell from basal area
#> Error:
#> ! object 'ba' not found
names(carbon) <- "carbon"
```

```
#> Error:
#> ! object 'carbon' not found
```

Logical operators work the same way and return a mask of ones and zeros. A “stocked” mask, cells carrying any measurable basal area, is a single comparison:

```
stocked <- ba > 0.05
#> Error:
#> ! object 'ba' not found
plot(stocked, main = "Stocked cells")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'stocked' not found
```

3.4 Reclassification

Turning a continuous surface into a small number of classes is reclassification, the operation behind every land-cover and zonation map. `classify()` takes a matrix of from-to-becomes rows and cuts the surface at the thresholds:

```
rc1 <- matrix(c(-Inf, 0.05, 1,      # sparse
               0.05, 0.30, 2,      # moderate
               0.30, Inf, 3),      # dense
             ncol = 3, byrow = TRUE)
zones <- classify(ba, rc1)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'classify': object 'ba' not found
levels(zones) <- data.frame(id = 1:3, class = c("sparse", "moderate", "dense"))
#> Error:
#> ! object 'zones' not found
plot(zones, main = "Stocking classes")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'zones' not found
```

3.5 Extraction

The bridge from raster back to field data is extraction: reading the raster value at each point location. `terra::extract()` samples the surface wherever a plot, a tree, or a sample sits, which is how gridded covariates enter a plot-level model:

```
pts <- spatSample(as.polygons(ext(ba), crs = crs(ba)), 5, method = "random")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'spatSample': error in evaluating the argument 'x' in selecting a method for function 'as.polygons': error in evaluating the argument 'x' in selecting a method for function 'ext': object 'ba' not found
extract(ba, pts)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'extract': object 'ba' not found
```

Extraction and reclassification are the two operations that connect the raster and vector worlds, and everything applied in the rest of the book uses one or both.

3.6 Keystone: land-cover reclassification

Remote-sensing verification turns imagery into a land-use / land-cover (LULC) map and then reasons about it: how much forest, how much converted, how the classes change through time. The Bolivia national land-cover work does this at country scale; the mechanics are the reclassification and area accounting shown here, which we later carry into the uncertainty chapter to attach a confidence interval to each class area.

We stand in for a spectral index with a smooth synthetic surface over the plot, classify it into cover types by threshold, then reclassify to the forest / non-forest split that most carbon accounting turns on:

```
set.seed(1)
xy <- xyFromCell(r, 1:ncell(r))
#> Error in `h()`:
#> ! error in evaluating the argument 'object' in selecting a method for function 'xyFromCell': object 'r' not found
val <- 0.55 + 0.30 * sin(xy[, 1] / 120) + 0.20 * cos(xy[, 2] / 160) +
  rnorm(ncell(r), 0, 0.05)
#> Error:
#> ! object 'xy' not found
ndvi <- setValues(r, pmin(pmax(val, -0.1, na.rm = TRUE), 0.95))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'setValues': object 'r' not found
names(ndvi) <- "ndvi"
#> Error:
#> ! object 'ndvi' not found

# thresholds -> cover classes
lulc <- classify(ndvi, matrix(c(-Inf, 0.2, 1,      # water/bare
                              0.2, 0.5, 2,      # grass/shrub
                              0.5, Inf, 3),      # forest
                             ncol = 3, byrow = TRUE))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'classify': object 'ndvi' not found
levels(lulc) <- data.frame(id = 1:3,
                           cover = c("water/bare", "grass/shrub", "forest"))
#> Error:
#> ! object 'lulc' not found
plot(lulc, main = "Land cover from a spectral index")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'lulc' not found
```

The class areas are the deliverable. `freq()` counts cells per class, and multiplying by the cell area gives hectares:

```
cell_ha <- prod(res(lulc)) / 1e4
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'res': object 'lulc' not found
f <- freq(lulc)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'freq': object 'lulc' not found
f$area_ha <- round(f$count * cell_ha, 2)
#> Error:
#> ! object 'f' not found
f[, c("value", "count", "area_ha")]
#> Error:
#> ! object 'f' not found
```

Reclassifying the three classes into a forest mask, the exact step behind a forest-area or deforestation figure, is one more `classify()`:

```
forest <- classify(lulc, matrix(c(1, 0, 2, 0, 3, 1), ncol = 2, byrow = TRUE))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'classify': object 'lulc' not found
forest_ha <- global(forest, "sum", na.rm = TRUE)[1, 1] * cell_ha
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'forest' not found
round(forest_ha, 1) # hectares of forest
#> Error:
#> ! object 'forest_ha' not found
```

That single number, forest area, is what a project claims and an auditor recomputes. Chapter 7 returns to it to ask how *certain* the number is, because a class area is only as good as the map's accuracy.

3.7 Exercises

1. Build the basal-area raster at 20-metre resolution instead of 10. How do the mean cell value and the number of cells change, and why does total basal area stay the same?
2. Reclassify the basal-area surface into four stocking classes and report the area in hectares of each. Which class covers the most ground?
3. Extract the basal-area value at the location of every stem with `extract()`. Why is the extracted value not the same as the stem's own basal area?
4. (Applied.) Change the forest threshold on the synthetic index from 0.5 to 0.45 and recompute forest area. By how many hectares does the estimate move, and what does that sensitivity imply for defining a class boundary in a real LULC map?

4. Terrain and Hydrology

4.1 Elevation

Terrain is the covariate behind almost everything else in a forest: it drives productivity, access, erosion, and where water collects. It arrives as a digital elevation model (DEM), a raster whose cell value is height above a datum. In operational work you download one, and the `elevatr` package fetches open tiles for any area:

```
library(elevatr)
aoi <- st_as_sf(data.frame(x = -78.143, y = 38.896),
  coords = c("x", "y"), crs = 4326)
dem <- get_elev_raster(aoi, z = 13) |> rast()
```

So the chapter runs offline and reproducibly, we build a DEM with a valley over the study plot instead. The surface tilts gently and is cut by a central channel, the kind of relief that governs drainage:

```
library(terra)
library(sf)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
  coords = c("lon", "lat"), crs = 4326) |>
```

```

st_transform(32617)

dem <- rast(ext(vect(stems)), resolution = 10, crs = "EPSG:32617")
#> Error:
#> ! [rast] empty srs
xy <- xyFromCell(dem, 1:ncell(dem))
#> Error in `h()`:
#> ! error in evaluating the argument 'object' in selecting a method for function 'xyFromCell': object 'dem' not found
ymid <- mean(xy[, 2])
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'mean': object 'xy' not found
elev <- 120 + 0.03 * (xy[, 1] - min(xy[, 1])) -
      18 * exp(-(xy[, 2] - ymid) / 70)^2)
#> Error:
#> ! object 'xy' not found
dem <- setValues(dem, elev)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'setValues': object 'dem' not found
names(dem) <- "elevation"
#> Error:
#> ! object 'dem' not found
global(dem, c("min", "max", "mean"))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'dem' not found

```

4.2 Terrain derivatives

Elevation is rarely the variable of interest; what it *implies*, slope, aspect, ruggedness, is. `terrain()` derives these directly. Slope and aspect need a projected DEM so horizontal and vertical distances share units, which ours already is:

```

slope <- terrain(dem, "slope", unit = "degrees")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'terrain': object 'dem' not found
aspect <- terrain(dem, "aspect", unit = "degrees")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'terrain': object 'dem' not found
tri <- terrain(dem, "TRI") # terrain ruggedness index
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'terrain': object 'dem' not found
plot(c(slope, tri), main = c("Slope (deg)", "Ruggedness (TRI)"))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'slope' not found

```

Slope matters for operability and erosion risk; ruggedness is a useful productivity covariate because sheltered, gentler terrain often carries taller, denser stands. A hillshade turns terrain into an intuitive relief image by simulating illumination, the standard base layer under any map:

```

hs <- shade(terrain(dem, "slope", unit = "radians"),
            terrain(dem, "aspect", unit = "radians"),
            angle = 40, direction = 315)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'terrain': object 'dem' not found
plot(hs, col = grey(0:100 / 100), legend = FALSE, main = "Hillshade")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'hs' not found

```

4.3 Flow direction

Water moves downhill, so a DEM already encodes drainage. `terrain()` computes flow direction, the neighbour each cell drains to, which is the first step of every hydrological routing algorithm and the input to catchment delineation:

```
flowdir <- terrain(dem, "flowdir")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'terrain': object 'dem' not found
plot(flowdir, main = "Flow direction")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'flowdir' not found
```

Full watershed delineation, flow accumulation, stream networks, catchment boundaries, is the domain of dedicated tools such as `whitebox`, which route flow across the accumulated grid. The principle is the one shown here: every hydrological product begins with elevation and the slope and flow fields derived from it.

4.4 Keystone: hydrological analysis

Wetland and inundation mapping asks a direct question of a DEM: given a water level, what land floods? The Lake Chilwa inundation work answers it across a shallow basin whose area swings enormously with small changes in level. The method is a threshold on elevation, and it is the same logic whether the water is a lake, a floodplain, or a reservoir behind a proposed dam.

We treat the valley floor as the basin. Flooded cells are those below a chosen water level, and the flooded area is the deliverable:

```
water_level <- min(values(dem), na.rm = TRUE) + 3 # 3 m above the channel floor
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'values': object 'dem' not found
flooded <- dem < water_level
#> Error:
#> ! object 'dem' not found
cell_ha <- prod(res(dem)) / 1e4
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'res': object 'dem' not found
flooded_ha <- global(flooded, "sum", na.rm = TRUE)[1, 1] * cell_ha
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'flooded' not found
round(flooded_ha, 2) # hectares inundated
#> Error:
#> ! object 'flooded_ha' not found

plot(hs, col = grey(0:100 / 100), legend = FALSE, main = "Inundation at +3 m")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'hs' not found
plot(flooded, add = TRUE, alpha = 0.5, col = c(NA, "steelblue"), legend = FALSE)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'flooded' not found
```

The value of the method is the rating curve: sweep the water level and record the area at each, and you have the relationship between stage and flooded extent that drives flood risk, wetland carbon, and reservoir planning:

```

levels <- seq(min(values(dem), na.rm = TRUE) + 1,
             min(values(dem), na.rm = TRUE) + 8, by = 1)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'values': object 'dem' not found
area_at <- sapply(levels, function(L)
  global(dem < L, "sum", na.rm = TRUE)[1, 1] * cell_ha)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'dem' not found
data.frame(water_level_m = round(levels, 1), flooded_ha = round(area_at, 1))
#> Error in `round()`:
#> ! non-numeric argument to mathematical function

```

Reading down the table, a few metres of stage change a small flooded area into a large one, the non-linearity that makes shallow basins so sensitive and makes stating the assumed water level essential to any wetland-area claim. The real project adds a satellite-derived water mask to validate the DEM prediction, but the analytical core is this thresholded elevation surface.

4.5 Exercises

1. Build the DEM and report its relief (max minus min elevation). Derive slope and describe where the steepest ground lies relative to the central valley.
2. Compute a hillshade with the sun in the northeast (`direction = 45`) and compare it with the northwest illumination used above. Why does illumination direction change which slopes look bright?
3. Recompute flooded area at water levels of +2, +4 and +6 m. Plot area against level: is the relationship linear? What does the shape tell you about the basin?
4. (*Applied.*) A project claims a 50-hectare seasonal wetland. Explain how you would use a DEM, a water-level assumption, and the rating-curve method above to test that claim, and what single assumption most affects your answer.

5. LiDAR and Point Clouds

5.1 Point clouds

Airborne LiDAR measures the forest in three dimensions. A scanner fires laser pulses and times their return, and each pulse can produce several returns as it passes through the canopy: the first off the treetops, later ones off branches and understory, the last off the ground. The result is a dense point cloud, millions of x, y, z points, from which we reconstruct both the ground surface and the vegetation above it. Full-waveform sensors record the entire returned energy profile rather than discrete points, giving even finer vertical structure. The dedicated R package is `lidR`, which reads the LAS/LAZ files LiDAR ships in.

We illustrate the vertical structure with a simulated transect: a sloping ground surface with canopy returns scattered above it, exactly the mix a single scan line produces.

```

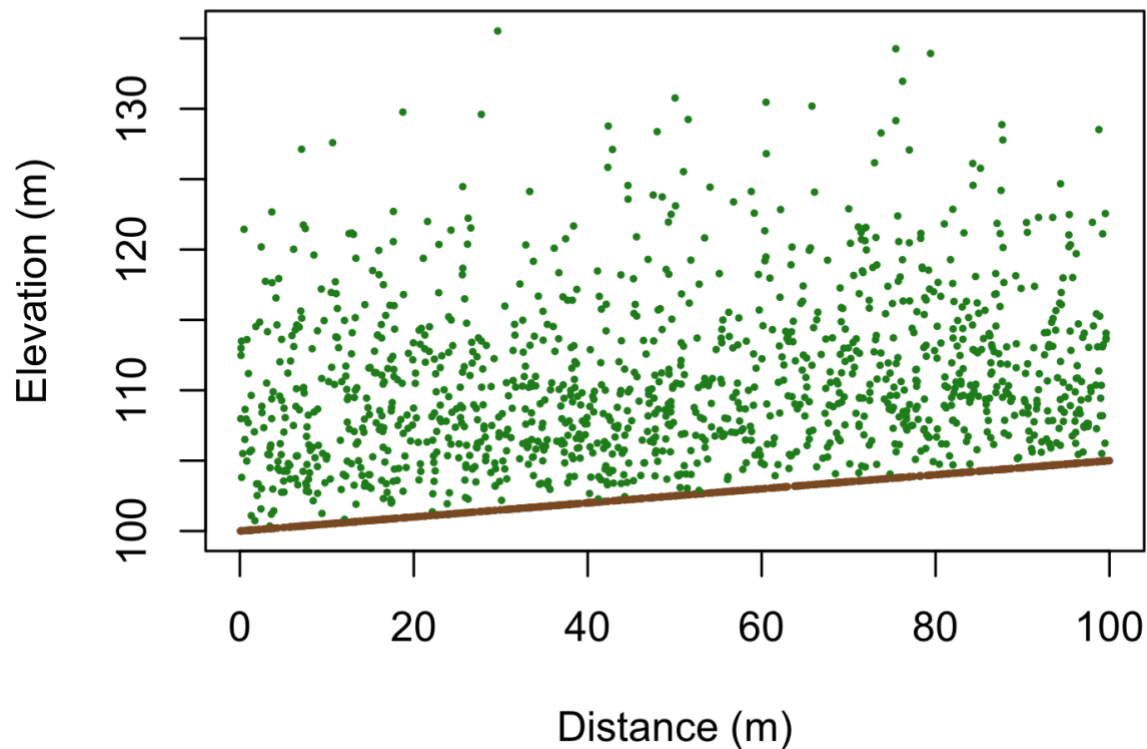
set.seed(1)
n <- 2000
x <- runif(n, 0, 100)
ground <- 100 + 0.05 * x
canopy <- runif(n) < 0.6
z <- ground + ifelse(canopy, rgamma(n, 2, 0.25), 0)

plot(x, z, pch = 20, cex = 0.4,
     col = ifelse(canopy, "forestgreen", "tan4"),

```

```
xlab = "Distance (m)", ylab = "Elevation (m)",  
main = "LiDAR returns: ground (brown) and canopy (green)")
```

LiDAR returns: ground (brown) and canopy (green)



5.2 Ground and canopy

Two surfaces come out of a point cloud. Classifying the lowest returns as ground and interpolating them gives the digital terrain model (DTM), the bare earth. Taking the highest returns gives the digital surface model (DSM), the top of the canopy. Their difference is the quantity forestry cares about, the canopy height model (CHM):

$$\text{CHM} = \text{DSM} - \text{DTM}$$

Ground classification is the delicate step, separating last returns that hit soil from those that hit low vegetation, and `lidR` implements the standard algorithms (cloth simulation, progressive morphological filters). The principle is that height above ground, not height above sea level, is what relates to biomass.

5.3 Building a CHM

Rather than process raw LAS here, we construct a CHM from the stem map, since each tree's height and crown define the canopy top. We give every stem a height from its diameter with a standard saturating growth curve, then rasterise the maximum height in each 5-metre cell:

```
library(terra)
library(sf)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
                  coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
stems$height <- 1.3 + 27 * (1 - exp(-0.035 * stems$dbh)) # DBH → height (m)

v <- vect(stems)
r <- rast(ext(v), resolution = 5, crs = "EPSG:32617")
#> Error:
#> ! [rast] empty srs
chm <- rasterize(v, r, field = "height", fun = max, background = 0)
#> Error in `h()` :
#> ! error in evaluating the argument 'y' in selecting a method for function 'rasterize': object 'r' not found
names(chm) <- "canopy_height"
#> Error:
#> ! object 'chm' not found
plot(chm, main = "Canopy height model (m)")
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'chm' not found
```

5.4 Tree detection

A CHM lets you find individual trees: each treetop is a local maximum, a cell higher than its neighbours. This is the basis of individual-tree detection, and the crown-radius-varying window in `ForestTools` refines it. We do the core operation directly, flagging cells that equal the maximum in a moving window and clear a minimum height:

```
win <- focal(chm, w = matrix(1, 5, 5), fun = max, na.policy = "omit")
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'focal': object 'chm' not found
tops <- (chm == win) & (chm > 5)
#> Error:
#> ! object 'chm' not found
tops[tops == 0] <- NA
#> Error:
#> ! object 'tops' not found
top_pts <- as.points(tops)
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'as.points': object 'tops' not found
nrow(top_pts) # detected treetops
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'nrow': object 'top_pts' not found

plot(chm, main = "Detected treetops")
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'chm' not found
plot(top_pts, add = TRUE, pch = 3, cex = 0.6)
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'top_pts' not found
```

5.5 Keystone: lidar-forestry

The lidar-forestry workflow chains exactly these steps into an inventory: classify ground, build the DTM and CHM, detect treetops, segment crowns, and derive canopy metrics that predict biomass. The reason it matters for verification is that a CHM covers every square metre, where field plots sample a fraction of a percent, so LiDAR turns a handful of plots into a wall-to-wall map, provided the plot-to-LiDAR relationship is calibrated and its error carried through.

The bridge to biomass is a canopy metric summarised per unit area and regressed on plot biomass. Mean canopy height is the simplest such metric:

```
cell_ha <- prod(res(chm)) / 1e4
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'res': object 'chm' not found
metrics <- data.frame(
  mean_canopy_height = round(global(chm, "mean", na.rm = TRUE)[1, 1], 1),
  p95_height         = round(quantile(values(chm), 0.95, na.rm = TRUE), 1),
  detected_trees     = nrow(top_pts),
  area_ha            = round(ncell(chm) * cell_ha, 1))
#> Error in `h()` :
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'chm' not found
metrics
#> Error:
#> ! object 'metrics' not found
```

Detected trees will fall short of the true stem count, because the understory hides beneath the canopy and only dominant trees form local maxima. That gap is the central caution of LiDAR inventory: it sees the canopy, not the stand, and the correction comes from calibrating against field plots that *do* count every stem. The height metrics, by contrast, are robust and are what a LiDAR biomass model is built on. The full project fits that model and maps predicted biomass with its prediction error, the subject of the uncertainty chapter.

5.6 Exercises

1. Rebuild the CHM at 2-metre resolution and describe how the number of detected treetops changes. Why does a finer grid tend to split single crowns into several maxima?
2. Raise the minimum-height threshold in the detection step from 5 m to 15 m. How does the detected count change, and which trees are you now excluding?
3. Compute the 25th, 50th, 75th and 95th height percentiles of the CHM. Explain why upper percentiles are better biomass predictors than the mean.
4. (*Applied.*) You have LiDAR over 10,000 ha and field plots on 30 ha. Describe, in steps, how you would use the plots to calibrate a CHM-based biomass model and produce a wall-to-wall estimate, and where the two main sources of error enter.

6. Disturbance and Risk

6.1 Change detection

The core remote-sensing task in verification is finding where the forest changed. The simplest and most transparent method is image differencing: take a spectral index at two dates and subtract. Where vegetation was lost, the index drops; a threshold on the difference marks candidate

disturbance. We build two dates of a synthetic index over the plot, the second with several patches cleared, and difference them.

```
library(terra)
library(sf)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
                  coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
r <- rast(ext(vect(stems)), resolution = 10, crs = "EPSG:32617")
#> Error:
#> ! [rast] empty srs

set.seed(7)
xy <- xyFromCell(r, 1:ncell(r))
#> Error in `h()`:
#> ! error in evaluating the argument 'object' in selecting a method for function 'xyFromCell': object 'r' not found

t1 <- setValues(r, 0.7 + 0.1 * sin(xy[, 1] / 100) + rnorm(ncell(r), 0, 0.03))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'setValues': object 'r' not found

# time 2: same forest, minus three cleared blocks
t2 <- t1
#> Error:
#> ! object 't1' not found
clear <- (xy[, 1] > 747050 & xy[, 1] < 747130 & xy[, 2] > 4306100 & xy[, 2] < 4306180) |
  (xy[, 1] > 747250 & xy[, 1] < 747300 & xy[, 2] > 4306300 & xy[, 2] < 4306380)
#> Error:
#> ! object 'xy' not found
t2[clear] <- 0.15 + rnorm(sum(clear), 0, 0.03)
#> Error:
#> ! object 'clear' not found
names(t1) <- "ndvi_t1"; names(t2) <- "ndvi_t2"
#> Error:
#> ! object 't1' not found

change <- t2 - t1
#> Error:
#> ! object 't2' not found
plot(change, main = "NDVI change (t2 - t1)")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'change' not found
```

A negative threshold isolates loss. The disturbed area is the immediate deliverable, and the same number that drives an emissions estimate:

```
disturbed <- change < -0.3
#> Error:
#> ! object 'change' not found
cell_ha <- prod(res(r)) / 1e4
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'res': object 'r' not found
dist_ha <- global(disturbed, "sum", na.rm = TRUE)[1, 1] * cell_ha
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'disturbed' not found

round(dist_ha, 2) # hectares of detected loss
#> Error:
#> ! object 'dist_ha' not found
```

6.2 Accuracy assessment

A map is a claim, and an unvalidated map is an unsupported claim. Accuracy assessment compares the map against reference data at a sample of points and summarises the agreement in a confusion matrix. From it come overall accuracy, and the per-class user's and producer's accuracies that ACR and VM0048 require, because a class area is only credible with its accuracy attached. We assess the disturbed / stable map against a reference sample:

```
set.seed(11)
ref <- data.frame(
  reference = sample(c("disturbed", "stable"), 120, replace = TRUE, prob = c(.3, .7))
# a map that is mostly right, with some confusion
ref$mapped <- ifelse(ref$reference == "disturbed",
  sample(c("disturbed", "stable"), 120, TRUE, c(.85, .15)),
  sample(c("disturbed", "stable"), 120, TRUE, c(.10, .90)))

cm <- table(mapped = ref$mapped, reference = ref$reference)
cm
#>           reference
#> mapped   disturbed stable
#> disturbed      20      7
#> stable         2     91

overall <- sum(diag(cm)) / sum(cm)
users   <- diag(cm) / rowSums(cm)      # commission: of mapped X, how many correct
producers <- diag(cm) / colSums(cm)    # omission: of true X, how many caught
round(c(overall = overall), 3)
#> overall
#> 0.925
round(rbind(users, producers), 3)
#>           disturbed stable
#> users      0.741  0.978
#> producers  0.909  0.929
```

Overall accuracy is the headline; the per-class figures are where problems hide. A high user's but low producer's accuracy for "disturbed" means the map misses real clearing, an omission error that understates loss, which is exactly the direction an auditor scrutinises. These same counts feed the area-adjusted estimator and its confidence interval in the next chapter.

6.3 Spatial risk

Some verification, additionality and permanence, turns on where disturbance is *likely*, not just where it happened. Risk modelling predicts that probability from spatial covariates. A robust and widely used predictor is proximity: disturbance begets disturbance, so distance to existing clearing is a strong risk covariate, the assumption behind deforestation-risk maps such as those in VM0048 baseline work. We compute distance to the detected disturbance and turn it into a risk surface:

```
disturbed[disturbed == 0] <- NA
#> Error:
#> ! object 'disturbed' not found
dist_to_loss <- distance(disturbed)      # metres to nearest clearing
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'distance': object 'disturbed' not found
```

```

risk <- exp(-dist_to_loss / 80) # decays with distance
#> Error:
#> ! object 'dist_to_loss' not found
names(risk) <- "risk"
#> Error:
#> ! object 'risk' not found
plot(risk, main = "Relative deforestation risk")
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'risk' not found

```

Classifying the surface into zones gives the map a planner or auditor acts on, and the area in each zone:

```

risk_zone <- classify(risk, matrix(c(-Inf, 0.2, 1,
                                   0.2, 0.5, 2,
                                   0.5, Inf, 3), ncol = 3, byrow = TRUE))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'classify': object 'risk' not found
levels(risk_zone) <- data.frame(id = 1:3, zone = c("low", "medium", "high"))
#> Error:
#> ! object 'risk_zone' not found
f <- freq(risk_zone); f$area_ha <- round(f$count * cell_ha, 1)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'freq': object 'risk_zone' not found
f[, c("value", "area_ha")]
#> Error:
#> ! object 'f' not found

```

6.4 Keystone: disturbance checks and VM0048 risk

Put together, the chapter is the disturbance-check pipeline: difference two dates to detect loss, quantify the area, validate it with an accuracy assessment, and model where further loss is likely. VM0048 and the associated risk tools formalise the last step into a baseline of expected deforestation; the ACR accuracy requirements formalise the middle step into a mandatory confusion matrix. The auditor's job is to reproduce each: recompute the change area, rebuild the confusion matrix from the reference sample, and check that the risk model's covariates and thresholds are defensible. Every number here, hectares lost, overall accuracy, area at high risk, is one a project reports and a verifier must be able to regenerate.

6.5 Exercises

1. Change the detection threshold from -0.3 to -0.2 and recompute disturbed area. Which way does the estimate move, and how does that trade omission against commission error?
2. From the confusion matrix, compute the omission and commission error for the "disturbed" class. Which type of error causes a project to *understate* emissions, and why does that concern a verifier most?
3. Rebuild the risk surface with a slower distance decay (divide by 160 instead of 80). How does the high-risk area change, and what real assumption does the decay rate encode?
4. (*Applied.*) A project reports 200 ha of avoided deforestation with an overall map accuracy of 78%. Explain why the accuracy figure alone is not enough, and which per-class accuracies you would demand before accepting the area.

7. Uncertainty

7.1 Why uncertainty

Every estimate in this book is wrong by some amount, and the discipline of verification is knowing by how much. A carbon figure without an uncertainty is not conservative, it is simply incomplete, and the crediting standards, ART-TREES, VCS, ACR, all deduct credits in proportion to how uncertain the estimate is. Reducing uncertainty is therefore worth real money, and quantifying it honestly is the auditor's core duty. This chapter builds the estimate, then its error, on the running stem data.

7.2 A carbon estimate

We convert diameters to aboveground biomass with a published generic hardwood equation (Jenkins et al., 2003), apply the IPCC default carbon fraction of 0.47 (IPCC, 2006), and scale to the plot area:

```
library(dplyr)
stems <- read.csv("data/scbi_stems.csv")

stems <- stems |>
  mutate(agb_kg = exp(-2.48 + 2.4835 * log(dbh)), # Jenkins generic hardwood
         carbon_Mg = 0.47 * agb_kg / 1000)

area_ha <- 25.6
total_C <- sum(stems$carbon_Mg)
C_per_ha <- total_C / area_ha
round(c(total_MgC = total_C, MgC_per_ha = C_per_ha), 1)
#>   total_MgC   MgC_per_ha
#>      82.3       3.2
```

That point estimate is the easy part. Three distinct sources of error sit beneath it: sampling error from measuring plots rather than the whole population, model error in the allometric equation, and parameter error in constants like the carbon fraction.

7.3 Sampling error

Divide the plot into quadrats and each becomes a sample of carbon density. The variation among them is the sampling uncertainty of the plot mean, summarised by the standard error and a confidence interval:

```
stems$quadrat <- paste(cut(stems$gx, 8), cut(stems$gy, 12))
quadrat_area <- 25.6 / (8 * 12) # ha per quadrat (equal-area cut)

qd <- stems |>
  group_by(quadrat) |>
  summarise(carbon_Mg = sum(carbon_Mg), .groups = "drop") |>
  mutate(carbon_ha = carbon_Mg / quadrat_area)

m <- mean(qd$carbon_ha); se <- sd(qd$carbon_ha) / sqrt(nrow(qd))
ci <- m + c(-1.96, 1.96) * se
```

```
round(c(mean = m, se = se, lower = ci[1], upper = ci[2]), 1)
#> mean se lower upper
#> 3.2 0.4 2.5 3.9
```

The half-width relative to the mean is the number the standards care about: the wider it is, the deeper the deduction.

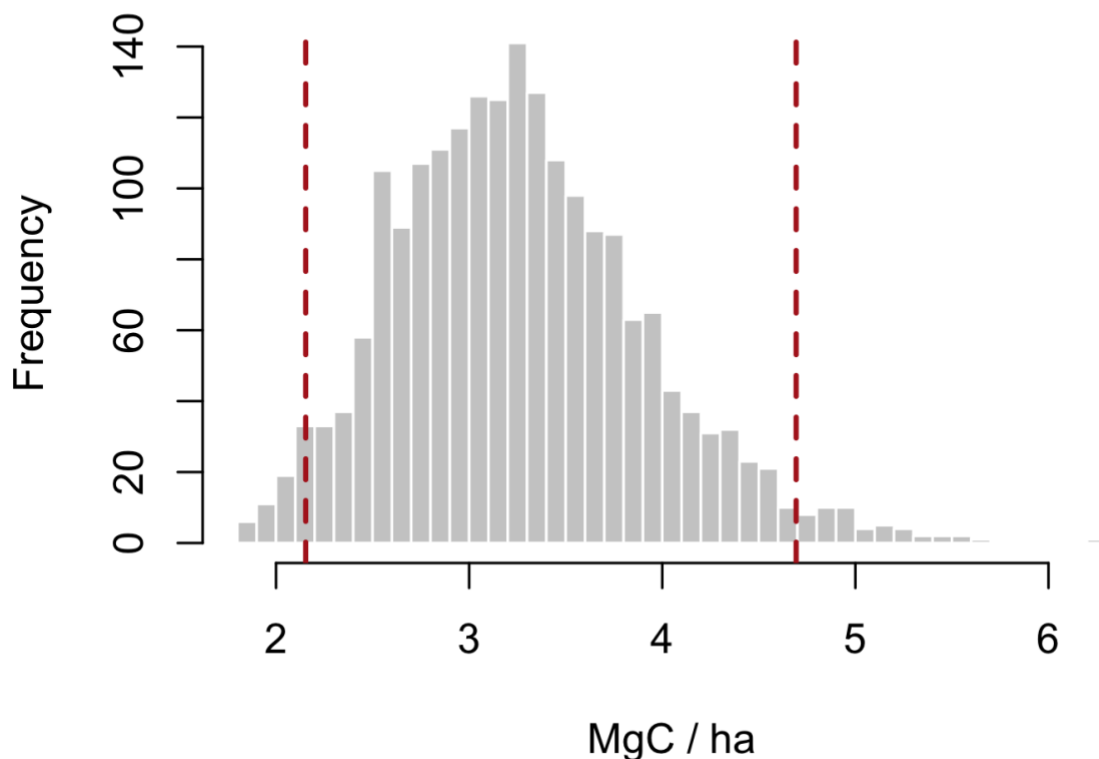
7.4 Monte Carlo propagation

Sampling error is only part of the story; the allometric model and the carbon fraction are uncertain too, and their errors compound. When error propagates through a non-linear function like $a D^b$, the clean way to combine sources is Monte Carlo simulation: draw the uncertain inputs from their distributions many times, recompute the whole estimate each time, and read the spread of results (IPCC, 2006).

```
set.seed(42)
n_sim <- 2000
sim <- numeric(n_sim)
for (i in seq_len(n_sim)) {
  b <- rnorm(1, 2.4835, 0.05) # allometric exponent uncertainty
  cf <- rnorm(1, 0.47, 0.02) # carbon-fraction uncertainty
  agb <- exp(-2.48 + b * log(stems$dbh))
  sim[i] <- sum(cf * agb / 1000) / area_ha
}
mc_ci <- quantile(sim, c(0.025, 0.975))
round(c(mean = mean(sim), lower = mc_ci[1], upper = mc_ci[2]), 1)
#> mean lower.2.5% upper.97.5%
#> 3.3 2.2 4.7

hist(sim, breaks = 40, col = "grey80", border = "white",
     main = "Monte Carlo carbon density", xlab = "MgC / ha")
abline(v = mc_ci, col = "firebrick", lwd = 2, lty = 2)
```

Monte Carlo carbon density



7.5 Conservative estimates

The standards translate uncertainty into a deduction. A common rule is that when the half-width of the 90% or 95% interval exceeds a threshold (often 15% of the mean), credits are issued against the conservative lower bound rather than the mean. Computing the uncertainty percentage and the conservative estimate is a few lines:

```
half_width <- (mc_ci[2] - mc_ci[1]) / 2
unc_pct <- 100 * half_width / mean(sim)
conservative <- mean(sim) - half_width # Lower-bound crediting basis
round(c(uncertainty_pct = unc_pct,
        mean = mean(sim), conservative = conservative), 1)
#> uncertainty_pct.97.5%      mean      conservative.97.5%
#>          38.9          3.3          2.0
```

If the uncertainty percentage clears the threshold, the gap between the mean and the conservative figure is credit the project forgoes for being imprecise, the direct financial incentive to measure better.

7.6 Keystone: area-adjusted accuracy

The Bolivia land-cover work raises the same question for areas rather than carbon: a class area read straight off a map inherits the map's classification errors, and good practice adjusts it and attaches a confidence interval (Olofsson et al., 2014). This is the LULC counterpart of the carbon deduction, and it uses the confusion matrix from the disturbance chapter. We take mapped forest and non-forest areas and an error matrix, and compute the area-adjusted estimate:

```
A <- c(forest = 18000, nonforest = 7000) # mapped area, ha
Atot <- sum(A); W <- A / Atot

# error matrix: rows = map class, cols = reference class
n <- matrix(c(95, 5, 12, 88), nrow = 2, byrow = TRUE,
            dimnames = list(map = names(A), ref = names(A)))
ni <- rowSums(n)

# Olofsson stratified estimator of the reference area proportion of "forest"
p_forest <- sum(W * n[, "forest"] / ni)
se_p <- sqrt(sum(W^2 * (n[, "forest"]/ni) * (1 - n[, "forest"]/ni) / (ni - 1)))

adj_forest_ha <- Atot * p_forest
ci_ha <- Atot * 1.96 * se_p
round(c(mapped_forest_ha = A["forest"],
        adjusted_forest_ha = adj_forest_ha,
        ci_halfwidth_ha = ci_ha), 0)
#> mapped_forest_ha.forest adjusted_forest_ha ci_halfwidth_ha
#> 18000 17940 893
```

The adjusted area differs from the mapped area because the map commits and omits forest at rates the reference sample reveals, and the confidence interval is the honest precision of the figure. A project that reports only the mapped area, with no adjustment and no interval, has reported a number no auditor can accept. The full Bolivia workflow applies this class by class across the national legend; the estimator is the one above.

7.7 Exercises

1. Recompute the sampling confidence interval using a 4-by-6 quadrat grid instead of 8-by-12. Does the mean change? Does the standard error? Explain the difference.
2. In the Monte Carlo, double the uncertainty on the carbon fraction (sd 0.04) and rerun. How much does the uncertainty percentage grow, and would it now trigger a 15% deduction?
3. Report the conservative (lower-bound) carbon density and express the forgone credit as a percentage of the mean. In one sentence, state the incentive this creates.
4. (*Applied.*) Change the error matrix so the map omits more forest (change the second row to 25, 75). Recompute the adjusted forest area and its interval, and explain which way omission error pushes the adjustment.

8. Time Series

8.1 Data cubes

Everything so far has been a snapshot. Monitoring is about change, and change lives in time, so the capstone data structure is the space-time cube: a stack of raster layers sharing one grid, indexed by date. A pixel is no longer a single value but a time series, and the questions become temporal, is

this pixel trending down, when did it drop, how fast is it recovering. We build a nine-year annual index cube over the plot, with a slow decline and a fire in 2021 that clears a block and holds it low.

```
library(terra)
library(sf)

stems <- st_as_sf(read.csv("data/scbi_stems.csv"),
                  coords = c("lon", "lat"), crs = 4326) |>
  st_transform(32617)
r <- rast(ext(vect(stems)), resolution = 10, crs = "EPSG:32617")
#> Error:
#> ! [rast] empty srs

years <- 2016:2024
set.seed(3)
xy <- xyFromCell(r, 1:ncell(r))
#> Error in `h()`:
#> ! error in evaluating the argument 'object' in selecting a method for function 'xyFromCell': object 'r' not found

burn <- xy[, 1] > 747100 & xy[, 1] < 747250 & xy[, 2] > 4306150 & xy[, 2] < 4306300
#> Error:
#> ! object 'xy' not found

cube <- rast(lapply(seq_along(years), function(k) {
  v <- 0.80 - 0.008 * (k - 1) + rnorm(ncell(r), 0, 0.015) # gentle decline
  if (years[k] >= 2021) v[burn] <- 0.20 + rnorm(sum(burn), 0, 0.02) # fire + no recovery
  setValues(r, v)
}))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'rast': error in evaluating the argument 'x' in selecting a method for function 'ncell': object 'r' not found
names(cube) <- paste0("y", years)
#> Error:
#> ! object 'cube' not found
nlyr(cube)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'nlyr': object 'cube' not found
```

8.2 Temporal reduction

A cube is reduced along time the way a single raster is summarised along space. `app()` applies a function to each pixel's time series. The mean over years is the baseline surface; the standard deviation flags where the record is unstable:

```
mean_ndvi <- app(cube, mean)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'app': object 'cube' not found
sd_ndvi <- app(cube, sd)
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'app': object 'cube' not found
plot(c(mean_ndvi, sd_ndvi), main = c("Mean index", "Temporal SD"))
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'mean_ndvi' not found
```

High temporal SD is the signature of disturbance: stable forest barely varies year to year, so the burned block lights up.

8.3 Trends

The direction and rate of change per pixel is the slope of index against year, a linear trend fitted independently at every location. `app()` carries the regression across the cube:

```
slope <- app(cube, function(v) {  
  if (all(is.na(v))) return(NA)  
  coef(lm(v ~ years))[2]  
})  
#> Error in `h()`:  
#> ! error in evaluating the argument 'x' in selecting a method for function 'app': object 'cube' not found  
names(slope) <- "trend_per_year"  
#> Error:  
#> ! object 'slope' not found  
plot(slope, main = "Per-pixel trend (index / year)")  
#> Error in `h()`:  
#> ! error in evaluating the argument 'x' in selecting a method for function 'plot': object 'slope' not found
```

Strong negative slopes mark sustained loss. This is the engine of trend-based monitoring: a whole landscape of time series reduced to one map of who is gaining and who is losing.

8.4 Detecting the event

A trend says a pixel declined; monitoring needs to know *when*. Differencing consecutive years isolates the step. The fire year is the layer where the largest drop occurs, and thresholding the year-on-year change delineates the burned area:

```
d2021 <- cube[["y2021"]] - cube[["y2020"]]  
#> Error:  
#> ! object 'cube' not found  
burned <- d2021 < -0.3  
#> Error:  
#> ! object 'd2021' not found  
cell_ha <- prod(res(r)) / 1e4  
#> Error in `h()`:  
#> ! error in evaluating the argument 'x' in selecting a method for function 'res': object 'r' not found  
burned_ha <- global(burned, "sum", na.rm = TRUE)[1, 1] * cell_ha  
#> Error in `h()`:  
#> ! error in evaluating the argument 'x' in selecting a method for function 'global': object 'burned' not found  
round(burned_ha, 2) # hectares burned in 2021  
#> Error:  
#> ! object 'burned_ha' not found
```

8.5 Keystone: IPCC Tier 1 fire emissions

The capstone joins the whole book: a time series detects a fire, the burned area is measured, and IPCC Tier 1 methods turn that area into an emission. The IPCC-wildfire-emissions workflow runs this chain from burn-area mapping through to emission factors, and the governing equation is the same one every fire inventory uses (IPCC, 2006):

$$L_{fire} = A \times M_B \times C_f \times G_{ef} \times 10^{-3}$$

where A is burned area, M_B the fuel biomass available, C_f the combustion factor, and G_{ef} the emission factor for the gas. With the burned area from the cube and IPCC default factors for temperate forest:

```
A <- burned_ha      # ha, from the time-series detection
#> Error:
#> ! object 'burned_ha' not found
MB <- 120           # tonnes dry matter per ha (fuel load)
Cf <- 0.45          # combustion factor (fraction consumed)
Gef <- 1569         # g CO2 per kg dry matter (IPCC default)

CO2_tonnes <- A * MB * Cf * Gef * 1e-3
#> Error:
#> ! object 'A' not found
round(c(burned_ha = A, CO2_tonnes = CO2_tonnes), 1)
#> Error:
#> ! object 'A' not found
```

That single number, tonnes of CO2 from the fire, is the end of the pipeline this course has built toward: geospatial data, cleaned and projected in the early chapters, turned into a detected, measured, and quantified emission with every step reproducible. In a real inventory each factor carries its own uncertainty, so the honest final move is to wrap this equation in the Monte Carlo of the previous chapter and report the emission with a confidence interval. The methods are all now in hand.

```
set.seed(9)
sim <- replicate(2000, {
  mb <- rnorm(1, 120, 20)
  cf <- rnorm(1, 0.45, 0.08)
  gef <- rnorm(1, 1569, 100)
  A * mb * cf * gef * 1e-3
})
#> Error in `FUN()`:
#> ! object 'A' not found
round(quantile(sim, c(0.025, 0.5, 0.975)), 0) # CO2 tonnes: Lower, median, upper
#> Error in `h()`:
#> ! error in evaluating the argument 'x' in selecting a method for function 'quantile': object 'sim' not found
```

8.6 Exercises

1. Reduce the cube with `app()` to the maximum index per pixel instead of the mean. Where does the maximum surface differ most from the mean, and why there?
2. Fit the per-pixel trend on the pre-fire years only (2016–2020). How does the burned block's slope compare with the full-record slope, and what does that say about mixing a step change into a linear trend?
3. Recompute burned area with a -0.2 threshold on the 2021 difference. How does the emission estimate move, and which term in the IPCC equation is it multiplying?
4. (*Applied.*) Using the Monte Carlo output, report the median CO2 and its 95% interval, then state which of the three factors (fuel load, combustion factor, emission factor) you would most want to measure locally to narrow the interval.

9. Spatial Patterns

9.1 Point patterns

Every earlier chapter analysed values measured at locations: carbon at a plot, elevation at a pixel, an index through time. Point-pattern analysis asks a different question, about the locations themselves. Are the events, tree stems, seedlings, detected clearings, spread evenly, scattered at random, or clustered together? The answer characterises the process that generated them, and it is the second capstone of this course: a whole branch of spatial statistics with its own tools. We use `spatstat`, whose `ppp` object holds a point pattern inside an observation window.

The running example here is a real regeneration survey: conifer seedlings mapped after the 2015 Mt Midgeley wildfire in the Darkwoods Conservation Area of the southern Selkirk Mountains, British Columbia (Murphy et al., 2026). We load the seedlings and build a `ppp` from one four-hectare plot, its bounding box serving as the window:

```
library(spatstat)
#> Error in `library()`:
#> ! there is no package called 'spatstat'

seed <- read.csv("data/darkwoods_seedlings.csv")
p3 <- subset(seed, plot == 3)
win <- owin(range(p3$x), range(p3$y))
#> Error in `owin()`:
#> ! could not find function "owin"
pp <- ppp(p3$x, p3$y, window = win)
#> Error in `ppp()`:
#> ! could not find function "ppp"
pp
#> Error:
#> ! object 'pp' not found
```

The printout reports the number of points and the window: a little over 1,600 seedlings in a 200 by 200 metre plot. That is the pattern whose structure we now interrogate.

9.2 Spatial randomness

The reference against which every pattern is judged is complete spatial randomness (CSR): points falling independently and with equal probability everywhere, a homogeneous Poisson process. Real patterns depart from CSR in one of two directions, toward regularity, where points avoid each other as in a planted stand, or toward clustering, where points attract as in natural regeneration around surviving seed trees. The whole toolkit exists to measure that departure and test whether it is real. A plot of the seedlings already suggests the answer:

```
plot(pp, main = "Mapped conifer seedlings, plot 3", pch = 20, cex = 0.3, cols = "forestgreen")
#> Error:
#> ! object 'pp' not found
```

9.3 Intensity

The intensity is the first-order property: the expected number of points per unit area, and how it varies across the window. A kernel density estimate smooths the points into a continuous intensity surface, the point-pattern analogue of a heat map, revealing where regeneration concentrated:

```
plot(density(pp, sigma = 8), main = "Seedling intensity (points/m²)")
#> Error:
#> ! object 'pp' not found
points(pp, pch = 20, cex = 0.2)
#> Error:
#> ! object 'pp' not found
```

Regeneration is patchy, not uniform: dense pockets sit near surviving seed sources and gentler, wetter ground, while other areas stay bare. That spatial unevenness is the signal, and explaining it is what the disturbance and terrain covariates in the study were for.

9.4 Nearest neighbour

Whether points cluster or repel is the second-order property, about the interaction between them. The most direct diagnostic is the nearest-neighbour distance: how far, on average, each seedling sits from its closest neighbour, compared with what CSR predicts. Under CSR the expected mean nearest-neighbour distance is $1/(2\sqrt{\lambda})$, so the ratio of observed to expected is the Clark–Evans index: below one means clustered, above one means regular.

```
ce_ratio <- function(df) {
  q <- ppp(df$x, df$y, window = win)
  obs <- mean(nndist(q))
  expected <- 1 / (2 * sqrt(intensity(q)))
  obs / expected
}
round(ce_ratio(p3), 3) # < 1 indicates clustering
#> Error in `ppp()`:
#> ! could not find function "ppp"
```

The ratio sits well below one: seedlings are closer to one another than randomness would allow, the quantitative signature of clustered regeneration.

9.5 Ripley's K

A single number cannot capture how clustering changes with scale, points may cluster tightly at short range yet disperse at long range. Ripley's K function describes the pattern across all distances at once, counting on average how many other points lie within a growing radius of each point, against the CSR expectation. Plotting the observed K with a simulation envelope is the standard test:

```
K <- envelope(pp, Kest, nsim = 19, verbose = FALSE)
#> Error in `envelope()`:
#> ! could not find function "envelope"
plot(K, main = "Ripley's K with CSR envelope")
```

```
#> Error:
#> ! object 'K' not found
```

Where the observed curve rises above the envelope the pattern is more clustered than random at that distance; below it, more regular. The seedlings sit above the envelope across the mapped range, confirming clustering is real and not an artefact of a few tight groups, and the scale-explicit view is why K, and its variance-stabilised form the L function, is the workhorse of the field.

9.6 Keystone: seedlings

The published study runs exactly this arc, then goes one step further (Murphy et al., 2026). Across 3,886 mapped seedlings on three four-hectare plots, point-pattern analysis found significant clustering for all seven conifer species, with intensity governed by fire severity, distance to seed source, topography, and wind. The lean diagnostics above reproduce its core finding on one plot: clustered regeneration, quantified by the Clark–Evans ratio and confirmed by Ripley’s K. Repeating the Clark–Evans ratio species by species shows the clustering is not confined to the dominant lodgepole pine but holds across the community:

```
spp <- c("Pinus_contorta", "Larix", "Picea", "Pseudotsuga")
sapply(spp, function(s) {
  q <- subset(p3, species == s)
  if (nrow(q) < 20) return(NA)
  round(ce_ratio(q), 3)
})
#> Error in `ppp()`:
#> ! could not find function "ppp"
```

Every species returns a ratio below one. The paper then models the intensity surface against the disturbance and terrain covariates with inhomogeneous Cox process models, showing that areas of overlapping beetle mortality and high-severity fire had significantly reduced pine regeneration. That covariate model is the natural next step beyond this book: the foundations here, CSR, intensity, nearest-neighbour, Ripley’s K, are what it is built on, and an auditor who understands them can read and challenge such a result.

9.7 Exercises

1. Build the `ppp` for plot 1 instead of plot 3 and compute its Clark–Evans ratio. Is the regeneration there clustered, random, or regular, and how does it compare with plot 3?
2. Map the intensity surface with a larger smoothing bandwidth (`sigma = 20`) and a smaller one (`sigma = 4`). How does the choice change what you see, and what does that imply about reporting a single heat map?
3. Run `envelope(pp, Kest)` on a random thinning of the seedlings (`rthin(pp, 0.2)`). Does the clustering signal survive when only a fifth of the points remain? What does that say about sample size and power?
4. (Applied.) Detected clearings in a monitoring image are also a point pattern. Explain how Ripley’s K could distinguish scattered natural tree-fall from an organised, clustered harvest front, and why that distinction matters for attributing disturbance.

Appendix A — Automation, Protocols and Registries

A.1 Beyond the analysis

A defensible result is only half the job. The other half is the discipline around it: making the work reproducible, governing the data to a recognised standard, and delivering credits into a registry that will accept them. This appendix sketches that surrounding practice. Its code is illustrative and is not run at render time.

A.2 Reproducible runtimes

The analysis in this book runs today; the test is whether it runs in three years, on another machine, for a reviewer who was not there. Two habits carry most of that weight. Pin your package versions so an update cannot silently change a result, and script the whole pipeline end to end so there are no undocumented manual steps.

```
# Pin the project's package library to exact versions
renv::init()           # snapshot the library
renv::snapshot()       # write renv.lock, committed with the project
renv::restore()        # a reviewer rebuilds the exact environment
```

A single entry point that runs every stage in order is the other half. When the raw data change, one command regenerates every figure and number, and nothing depends on remembering which script to run first.

```
# run.R — the one command that rebuilds everything
source("01-ingest.R")    # read and validate raw survey data
source("02-process.R")  # projection, extraction, modelling
source("03-report.R")    # render outputs with quarto::quarto_render()
```

A.3 Data protocols and QA/QC

Every number inherits the quality of the survey that produced it. Standardised digital capture, enforced schemas, and a logged accuracy estimate on every GPS fix are what let an auditor trust a plot table at all. The governing reference is the geospatial data-quality framework of the ISO 19100 series, which defines the quality elements, completeness, logical consistency, positional accuracy, thematic accuracy, an auditor should expect to see documented. In practice this means a QA/QC step that checks geometry validity, coordinate ranges, attribute domains, and duplicate records before any analysis begins.

```
# Minimal ingest QA gate
stopifnot(
  all(sf::st_is_valid(layer)),           # geometry validity
  all(sf::st_coordinates(layer)[, 1] > -180), # coordinates in range
  !anyDuplicated(layer$plot_id)         # no duplicate keys
)
```

Appendix B — References

- Bivand, R. S., Pebesma, E., & Gómez-Rubio, V. (2013). *Applied spatial data analysis with r* (2nd ed.). Springer.
- Hijmans, R. J. (2024). *Terra: Spatial data analysis*. <https://CRAN.R-project.org/package=terra>
- IPCC. (2006). *2006 IPCC guidelines for national greenhouse gas inventories, volume 4: Agriculture, forestry and other land use*. Institute for Global Environmental Strategies.
- Jenkins, J. C., Chojnacky, D. C., Heath, L. S., & Birdsey, R. A. (2003). National-scale biomass estimators for united states tree species. *Forest Science*, 49(1), 12–35. <https://doi.org/10.1093/forestscience/49.1.12>
- Lovelace, R., Nowosad, J., & Muenchow, J. (2019). *Geocomputation with r*. Chapman; Hall/CRC.
- Murphy, S., Leslie, A., Wilson, J., & Banks, L. K. (2026). Spatial patterns of conifer regeneration following mountain pine beetle outbreak (*dendroctonus ponderosae*) and mixed-severity wildfire: A point process model of disturbance interactions in the southern selkirk mountains, british columbia. *Forest Ecology and Management*, 618, 123985. <https://doi.org/10.1016/j.foreco.2026.123985>
- Olofsson, P., Foody, G. M., Herold, M., Stehman, S. V., Woodcock, C. E., & Wulder, M. A. (2014). Good practices for estimating area and assessing accuracy of land change. *Remote Sensing of Environment*, 148, 42–57. <https://doi.org/10.1016/j.rse.2014.02.015>
- Pebesma, E. (2018). Simple features for r: Standardized support for spatial vector data. *The R Journal*, 10(1), 439–446. <https://doi.org/10.32614/RJ-2018-009>
- Pebesma, E., & Bivand, R. (2023). *Spatial data science: With applications in r*. Chapman; Hall/CRC. <https://doi.org/10.1201/9780429459016>